

flint front ends

Four ways to put an S3 bucket in front of a workload — **flint-lite**, **flint-lean**, **flint-passthrough** and **flint-forge** — with the multi-cluster and security consequences of each, the one durable store they all share, and how they compose.

Scope. The architecture of the four front ends as built, the fleet shape each implies when one user launches agents across several Kubernetes clusters, what identity is actually enforced on the data path, the security posture of each, where S3 sits in all four, and the rule under which they can be used together. Every claim here is drawn from the repository at the revision noted below; where a guarantee has a stated ceiling, the ceiling is stated rather than rounded up, and where a drill refuted a claim the drill wins.

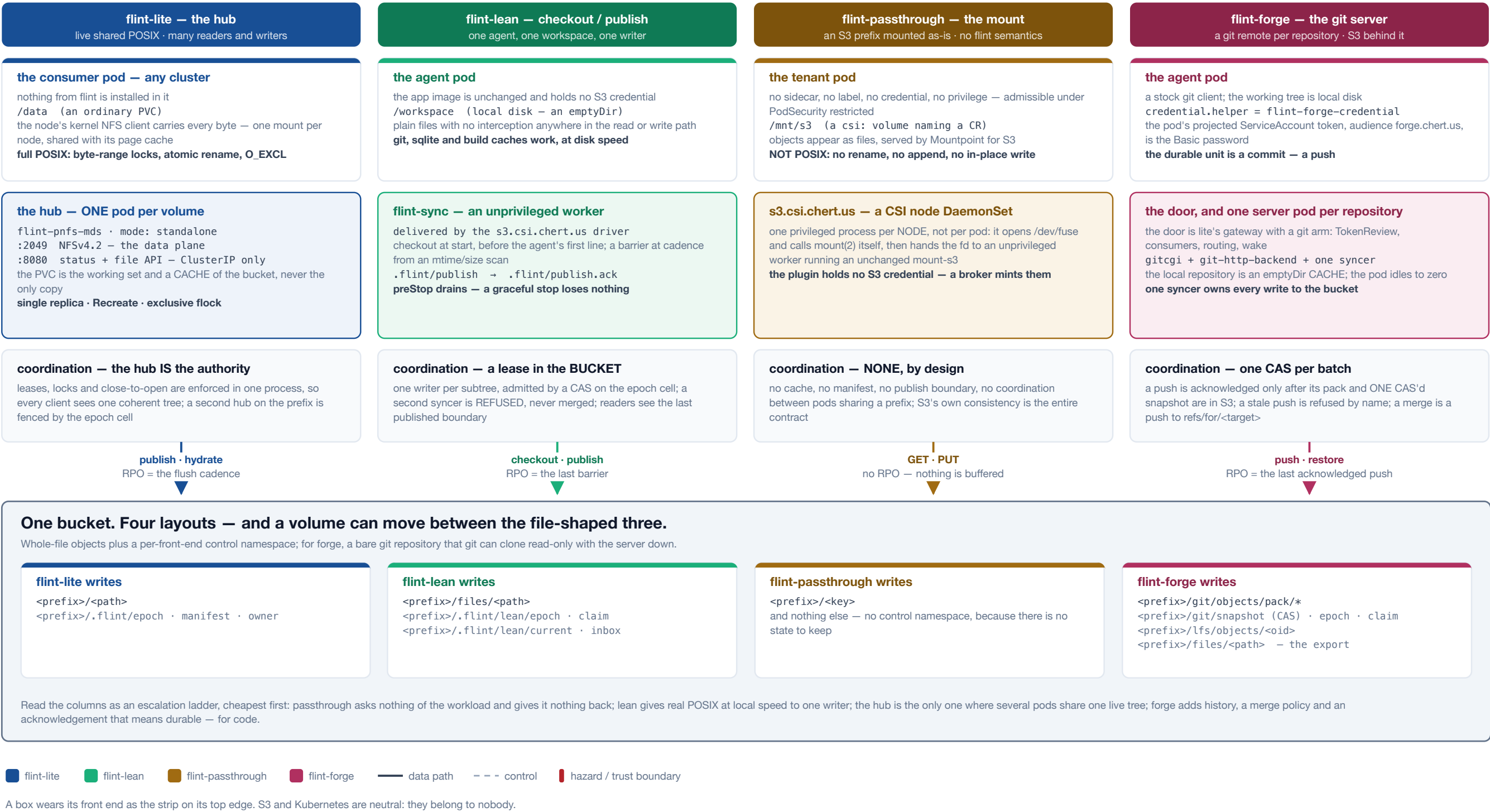
Sources of record. `docs/plans/csi-node-mount-design.md` (the CSI delivery), `docs/plans/flint-lean-plan.md`, `docs/flint-lite-architecture.html`, `docs/flint-lean-architecture.html`, `docs/plans/flint-forge-design.md` (§13 for the falsifiers as run, §17 for composition), the four charts, the drills under `forge/e2e/`, and the code under `spdk-csi-driver/src/{s3csi,passthrough,tier,lite_operator,lean_operator,forge_operator,lite_gateway}`, `lean/sidecar/src` and `forge/syncer/src`. The security page also draws on the approach radar's verified Security cells (`docs/radar/`).

Regenerating. `docs/architecture/diagrams.py` writes every diagram; `docs/architecture/build.sh` validates, rasterises and renders. The diagrams are standalone SVG files under `diagrams/`, referenced rather than inlined, so they can be reused in any other format; the build also emits a Markdown rendition for conversion. Colour means one thing in them: which front end — the same four hues the approach radar uses.

- 1 • Four front ends, one bucket** — the shape of the choice, and what is common underneath it.
- 2 • flint-lite — the hub** — one pod as the coherence authority, a PVC as cache, S3 as the copy.
- 3 • flint-lite across clusters** — one hub, many clusters, and the three things the fleet must supply.
- 4 • flint-lean — checkout and publish** — plain local files, one writer, a boundary the agent declares.
- 5 • flint-passthrough — the CSI mount** — a prefix mounted as-is, and where the privilege went.
- 6 • flint-forge — the git server** — real git per repository, one syncer, and a push that is acknowledged only once it is in S3.

- 7 • Where a user's token can and cannot go** — what each front end actually authenticates.
- 8 • Fleet shape and blast radius** — what you install per cluster, and what one compromised pod reaches.
- 9 • S3 — the durable store** — the live view, the published view, and the recovery point of each.
- 10 • Security posture** — seven questions, four columns, and where each boundary is actually drawn.
- 11 • Composition** — one pod, several doors, one bucket, one writer per prefix — and what the drills found.
- 12 • Choosing** — ten dimensions, four columns, one door per data class.

flint puts a bucket in front of a workload four different ways. They are not four configurations of one thing and not four tiers of the same product: they are four different bargains, each coherent with itself, over one bucket. Three of them are file-shaped and share a format; the fourth is git-shaped, and stores a bare repository. What varies is how much the workload gets and how much machinery has to exist for it to get that.



One bucket. Four layouts — and a volume can move between the file-shaped three.

Whole-file objects plus a per-front-end control namespace; for forge, a bare git repository that git can clone read-only with the server down.

flint-lite writes

<prefix>/<path>
<prefix>/.flint/epoch · manifest · owner

flint-lean writes

<prefix>/files/<path>
<prefix>/.flint/lean/epoch · claim
<prefix>/.flint/lean/current · inbox

flint-passthrough writes

<prefix>/<key>
and nothing else — no control namespace, because there is no state to keep

flint-forge writes

<prefix>/git/objects/pack/*
<prefix>/git/snapshot (CAS) · epoch · claim
<prefix>/lfs/objects/<oid>
<prefix>/files/<path> — the export

Read the columns as an escalation ladder, cheapest first: passthrough asks nothing of the workload and gives it nothing back; lean gives real POSIX at local speed to one writer; the hub is the only one where several pods share one live tree; forge adds history, a merge policy and an acknowledgement that means durable — for code.

flint-lite

flint-lean

flint-passthrough

flint-forge

— — — data path

- - - control

hazard / trust boundary

A box wears its front end as the strip on its top edge. S3 and Kubernetes are neutral: they belong to nobody.

Read the columns as an escalation ladder, cheapest first. Passthrough asks nothing of the workload — nine lines of pod spec — and gives it nothing back beyond the objects: no POSIX, no coordination, no boundary. Lean gives a real local filesystem at disk speed to one writer, and a durable point that writer can name. The hub is the only one where several pods, in several clusters, can share one live tree with locks that mean something. Forge stands slightly apart: it is for code, its unit of durability is a commit, and what it adds is history, a merge policy the server enforces, and an acknowledgement that means the bytes are in S3.

The common floor is the bucket format, and it is what makes the file-shaped choice reversible. Lite, lean and passthrough all write whole-file objects; what differs is the .flint/ control namespace beside them, which exists to answer questions the object listing cannot — who holds the write lease, what the last coherent boundary contained, what the whole tree is for a restore. Passthrough writes no control namespace at all, which is exactly why it can be pointed at a prefix somebody else's tooling already owns. Forge's bucket is a bare git repository plus one CAS'd pointer, and it joins the family through its export: a ref's tree published, by the shipped lean binary, as a valid lean workspace. Because the file formats are common and mutually convertible, a volume that outgrows one front end can move to another rather than being re-plumbed — and page 11 shows that the better question is usually not which one, but which one for which data.

One pod serves real POSIX over NFSv4.2 to any number of pods in any number of clusters. The PVC beside it is a working set; an S3 prefix is the durable copy. A second listener exposes the same filesystem over HTTP for callers that cannot mount. One hub serves exactly one prefix — a project that needs several volumes runs several hubs.



What the hub is for, and what it costs

It is the coherence authority

One process holds every lease, lock and open, so N pods across N clusters see ONE tree — the only front end where two writers share a directory live. That authority is the constraint: one pod, one replica, Recreate. A second hub on the prefix would be split-brain, which the epoch cell fences and the operator refuses up front. **One hub serves one prefix. Several volumes ⇒ several hubs.**

S3 is the durability; the RPO is the cadence

The PVC is a working set the hub flushes to the bucket on a timer. Everything durable is in S3; everything fast is on disk. A cold read hydrates first and both doors say so rather than blocking: NFS parks the caller with NFS4ERR_DELAY, HTTP answers 503 + Retry-After. **Losing the PVC is a rebuild. Losing the bucket is a loss.**

The two doors are secured differently

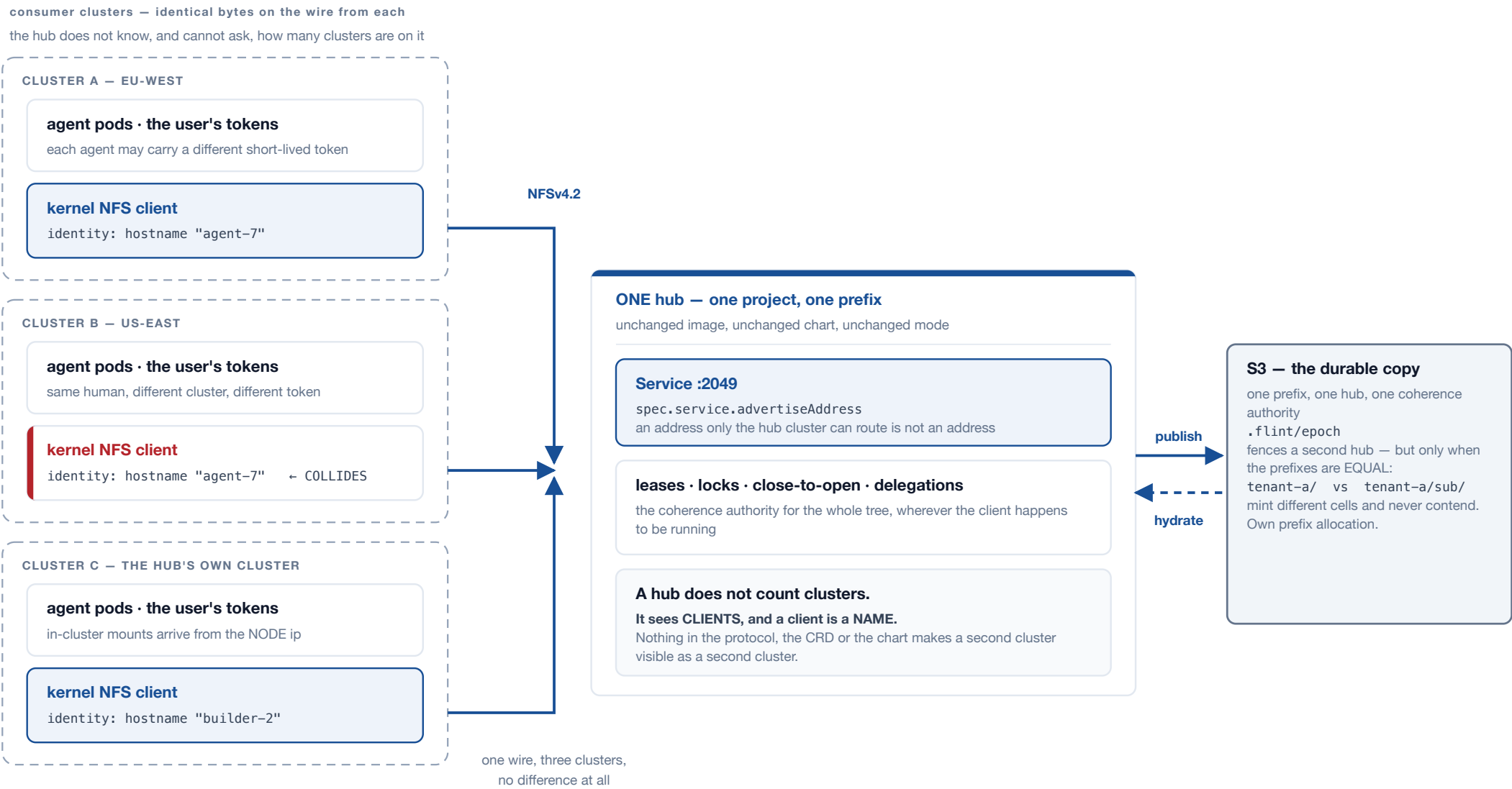
:2049 is AUTH_SYS — the client asserts its own uid and the server takes it, so reachability is the real boundary. :8080 can rewrite every file in the share, so it stays ClusterIP-only, never joins the Service, and mounts no routes at all without a bearer token — 404, not 401. **That token is per-SHARE, not per-user. See the identity page.**

— data path - - - control ■ hazard / trust boundary

The hub's value and its constraint are the same fact. One process holds every lease, lock and open, so N pods across N clusters see one coherent tree — and that is why it is one pod, one replica, Recreate, behind an exclusive flock. Two hubs on one prefix would be split-brain, which the store-side epoch cell fences and the operator refuses up front. Consumers install nothing: the data path is the node's own kernel NFS client, one mount per node, shared by every pod on it along with its page cache.

The disk is a cache; the bucket is the copy. The hub flushes the working set to S3 on a cadence, so the recovery point is that cadence and losing the PVC is a rebuild rather than a loss. A cold read hydrates on demand and both doors say so rather than hanging — NFS parks the caller with NFS4ERR_DELAY, HTTP answers 503 + Retry-After. **The two doors are secured differently, and only one of them has a credential.** Port 2049 is AUTH_SYS, so the client asserts its own uid and reachability is the real boundary; port 8080 can rewrite every file in the share, so it stays ClusterIP-only, never joins the Service, and mounts no routes at all without a bearer token — 404, not 401. That token is per-share, not per-user, which page 7 takes up.

One hub can serve agents in several clusters at once. The wire is ordinary NFSv4.2 and the hub does not know, or care, how many clusters are on it — which is precisely the problem. Three things the protocol and the chart cannot supply become the fleet's responsibility, and every one of them fails silently if it is missed.



What the FLEET must supply

three things neither the protocol nor the chart can do for you

1 A unique name per client

The identity is the hostname and nothing else. Two clusters sharing a node name are ONE client, and RFC 8881 says the server MUST read the second as the first REBOOTING — so the incumbent's locks and opens are dropped, silently, correctly, on a false premise.

hostname = <cluster>--<node>
nfs.nfs4_unique_id=<cluster>

2 A boundary at the network layer

Port 2049 is AUTH_SYS, so reachability IS the boundary — and networkPolicy cannot draw it: kube-proxy SNATs a NodePort or LB client to an address in the HUB's cluster before the packet arrives.

Measured: 1486 of 1486 connections from two remote clusters arrived as the hub cluster's gateway — none as a remote node.

Use peering, security groups or a gateway.

3 A keepalive per remote mount

The idle ladder suspends a quiet share, and a partitioned cluster's busy agents look exactly like quiet ones. Drive POST /wake from the thing that knows whether work is happening.

suspendWithSessions DEFAULTS to suspending anyway.

The image is unchanged. The operating contract is not.

Admission uniqueness stops at the cluster edge

The operator refuses a second share on one prefix from its own cache — CEL and policy see one object at a time. Two clusters each admit a share on one prefix and neither sees the other; the store-side epoch is the only backstop, and it catches EXACTLY equal prefixes.

Addresses are not arbitrated either

Two shares may advertise the same address and both report Conflict=False (Unique) — that condition covers bucket-prefix overlap. A true statement about STORAGE that says nothing about addresses.

The failure mode is silent by construction

A colliding client gets NO error. The protocol says it is the same client returning, so the server does exactly the right thing with a false premise, and the loser's state is simply gone.

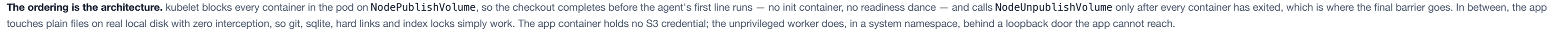
Six defects in v1.37.0 traced to this one cause.

— data path - - - control ■ hazard / trust boundary

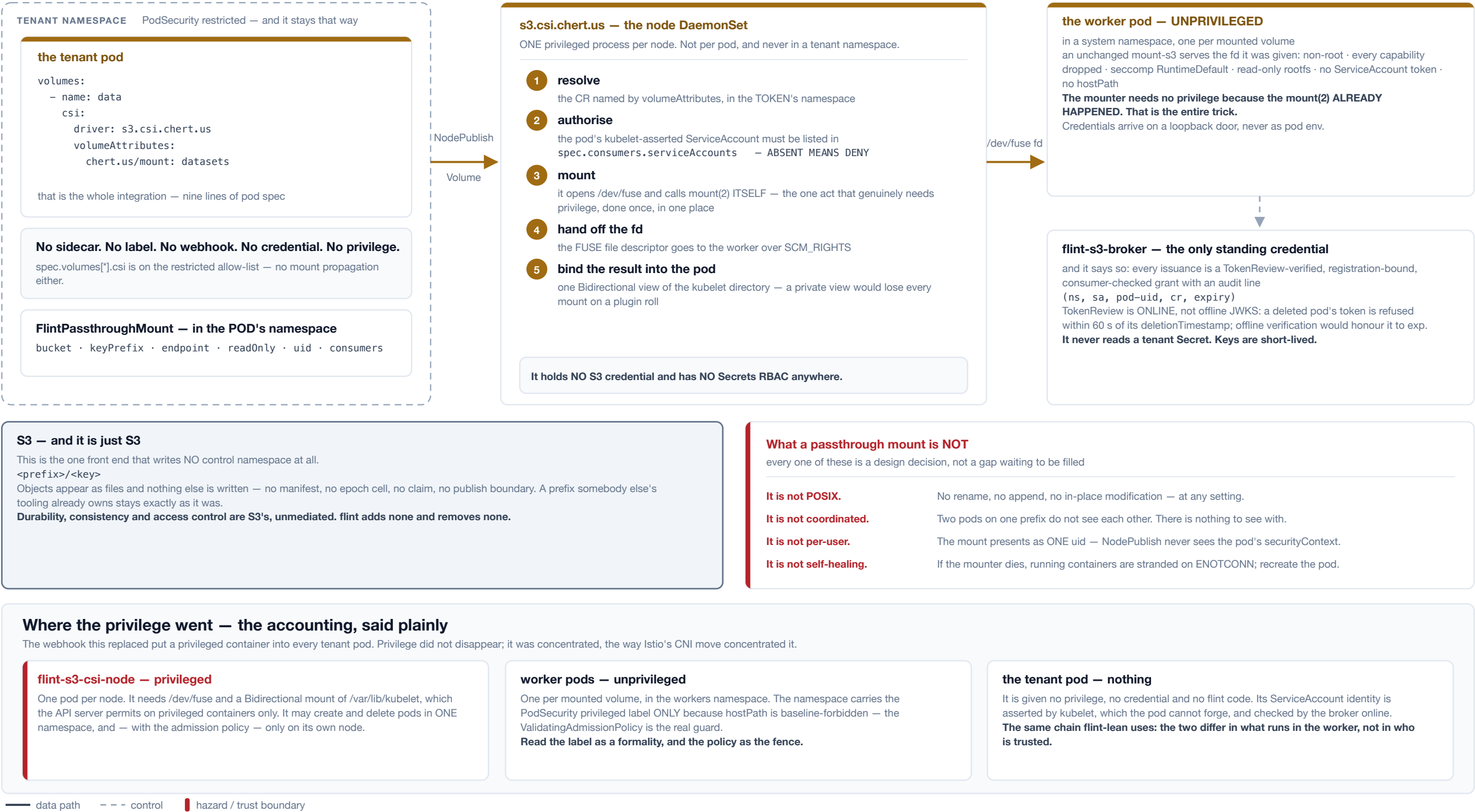
A hub does not count clusters — it sees clients, and a client is a name. The NFSv4 client identity is the hostname and nothing else, so two clusters that share a node name are one client to the hub, and RFC 8881 §18.35.5 requires the server to read the second as the first rebooting: the incumbent's locks and opens go, silently, correctly, on a false premise. Six defects in v1.37.0 traced to this one cause. Fix it with hostname = <cluster>--<node>, or the client-side kernel parameter nfs.nfs4_unique_id, which survives a rename.

The reachability boundary cannot be drawn where you would expect to draw it. kube-proxy SNATs a NodePort or LoadBalancer client to an address in the hub's own cluster before the packet arrives: measured on a three-cluster rig, 1486 of 1486 connections from two remote clusters arrived as the hub cluster's gateway, none as either remote node. Listing remote CIDRs in networkPolicy is an outage; listing the local one admits everyone. Use peering, security groups, or a gateway. And a partitioned cluster's busy agents look exactly like idle ones — suspendWithSessions defaults to suspending anyway, and even when set it only narrows the window, because leases expire. Drive POST /wake from the thing that actually knows whether work is happening.

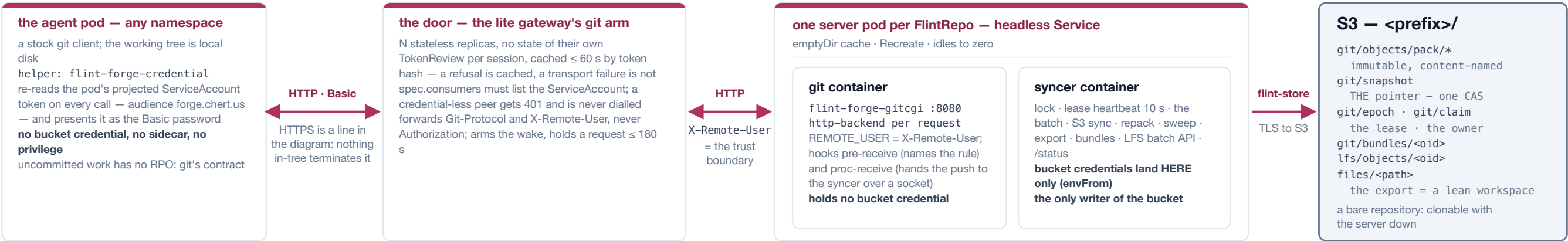
The **boundary verbs are files, because the workspace is the interface**. An agent that can write a file can declare a coherent point — `echo > .flint/publish` — and learn when its bytes are durable, from `.flint/publish.ack:ok` means the bytes are in S3, `refused-fenced` means this syncer lost the lease and is saying so rather than leaving the agent waiting forever. No client library, no credential, no network path. **Gated mode answers a different question from cadence**: cadence asks how stale a reader may be, gated asks whether a reader may see half a logical change — and answers no, by uploading every changed file as a new version immediately (durable, and invisible) and installing the whole pending set with one CAS. Gated is refused without a lag bound, so unbounded staleness is impossible by construction rather than by convention.



An S3 prefix, mounted into a pod as-is by Mountpoint for S3, with no flint semantics layered over it. There is no controller and no status, because nothing about a passthrough mount converges. What there is, is a careful answer to the question of who holds privilege — and the answer is: not the tenant.

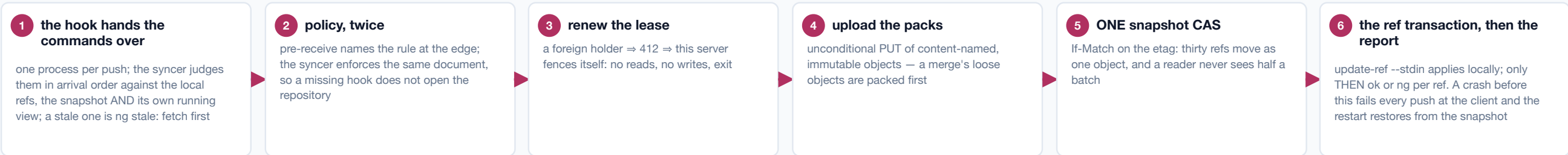


A git server per repository, with S3 as its only durable state. Agent pods are stock git clients; a stateless door turns the pod's own ServiceAccount token into a principal and routes to the repository's server pod, where real git — `git-http-backend` behind a small CGI runner of flint's, the standard hooks — serves from a local disk that is a cache, and one syncer owns every write to the bucket. The design's rule was to reinvent nothing git already does; what flint supplies is exactly what git lacks in a cluster with an object store behind it.



The push — acknowledged means durable

receive-pack serialises nothing and, under `proc-receive`, checks nothing: the syncer re-implements staleness and fast-forward, and batches every push that arrives while one is in flight.



Idle to zero — one rung, and a wake the door holds

idle for `suspendAfterSecs` $\Rightarrow$ replicas 0 and the `emptyDir` is gone, so a parked repository costs nothing but its objects. The ladder suspends only on the server's own polled `/status`, and a poll failure holds forever. The door arms `chert.us/requested-at` on the CR, waits on the CR — never the pod — and holds the request up to 180 s; the restore is one pack. **As built the clock counts pushes only: a fetch never reaches the syncer, so read-only use is parked one threshold after its wake.**

Fleet levers — take the server out of the transfer

A thousand clones are ~ 130 CPU-s but 43 GB from one NIC: egress binds before CPU. Bitmaps ship with every pack; a clone BUNDLE is cut on a floor, uploaded beside the packs and advertised as a presigned URL re-signed at half its TTL — the client must opt in (`transfer.bundleURI=true`). **Measured on EC2 (F8): 1,000 clones cost the server 5.7 MiB of egress with bundles advertised and the client opted in, 40,409 MiB with the opt-in off — 7,000x.** LFS: the batch API runs in the syncer and answers with presigned URLs, so a checkpoint goes client-to-store and the pod sees a few hundred bytes of JSON. The pruner takes only branches main already contains, and only once quiet.

The export — a legible mirror, by the shipped lean binary

After a batch moves an exported ref, the syncer materialises the tree with a two-tree read-tree — touching exactly what changed, removing what is gone — and runs `flint-sync` barrier over it, so lean's ordering (upload, CAS, deletes LAST) is inherited, not re-derived. It runs AFTER the report and never CASes the snapshot; lean and passthrough readers mount main read-only with no forge code in them. **A mirror that is never repaired: a foreign write into it is not noticed (the barrier diff's a local scan against a local baseline); a reader that verifies against the manifest refuses it, a mount with no manifest takes it. Readers of an export are readers.** It is awaited inline with the heartbeat: a blocked barrier now times out and backs off, but pushes stall for that long.

Built, and drilled on a real cluster — 2026-09-04

All eleven falsifiers ran on EC2 and went green: acknowledged-means-durable under 8 mid-push kills; two pushes to one ref; a merge surviving a cold restore; the fence deposing a straggler within one heartbeat; a byte-identical restore and a dumb clone from the bucket with the server down; protected main, which found a fresh repository unable to create main at all; idle-to-zero, with an 11 s clone through the door on a pod the request created; the storm above; the export, 3 of 164 objects rewritten by a three-file push, after a defect that froze it; the sweep; and an S3 outage — clones served, pushes refused, not ready in ~ 20 s. Phase 0 measured the buy option: Gitea's 4,019 MiB of egress for 100 clones sits level with forge's control arms. The composition drills, on a local MinIO rig, found the five things on the composition page. Not drilled: agents in another cluster. What forge is not: a web UI, pull requests, code search, CI, a POSIX volume, untracked files. Uncommitted work is not durable, which is git's contract; `.env`, build output and datasets are durable only if committed or put in LFS.

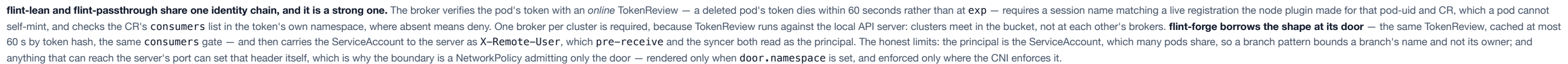
— data path - - - control █ hazard / trust boundary

Acknowledged means durable, and the mechanism is one syncer per repository. The review's central finding was that `receive-pack` serialises nothing: under `proc-receive` git performs neither the old-oid check nor the fast-forward test for the commands it hands off, so two concurrent pushes to one ref run their hooks fully overlapped. The syncer therefore re-implements both checks — against the local ref, the bucket's snapshot *and* its own running view of the batch — renews the lease, uploads the packs, CASes the one mutable object (`git/snapshot`, which names every ref, pack and bundle) and applies the ref transaction, and only then reports `ok` or `ng` per ref. A stale push is refused by name; a merge is a push to `refs/for/<target>` that the server performs with `merge-tree`; a crash before the report fails every push in the batch at the client, and the restart restores from the snapshot. Policy is enforced twice from one document — `pre-receive` names the rule, the syncer enforces it — so a missing hook does not open the repository.

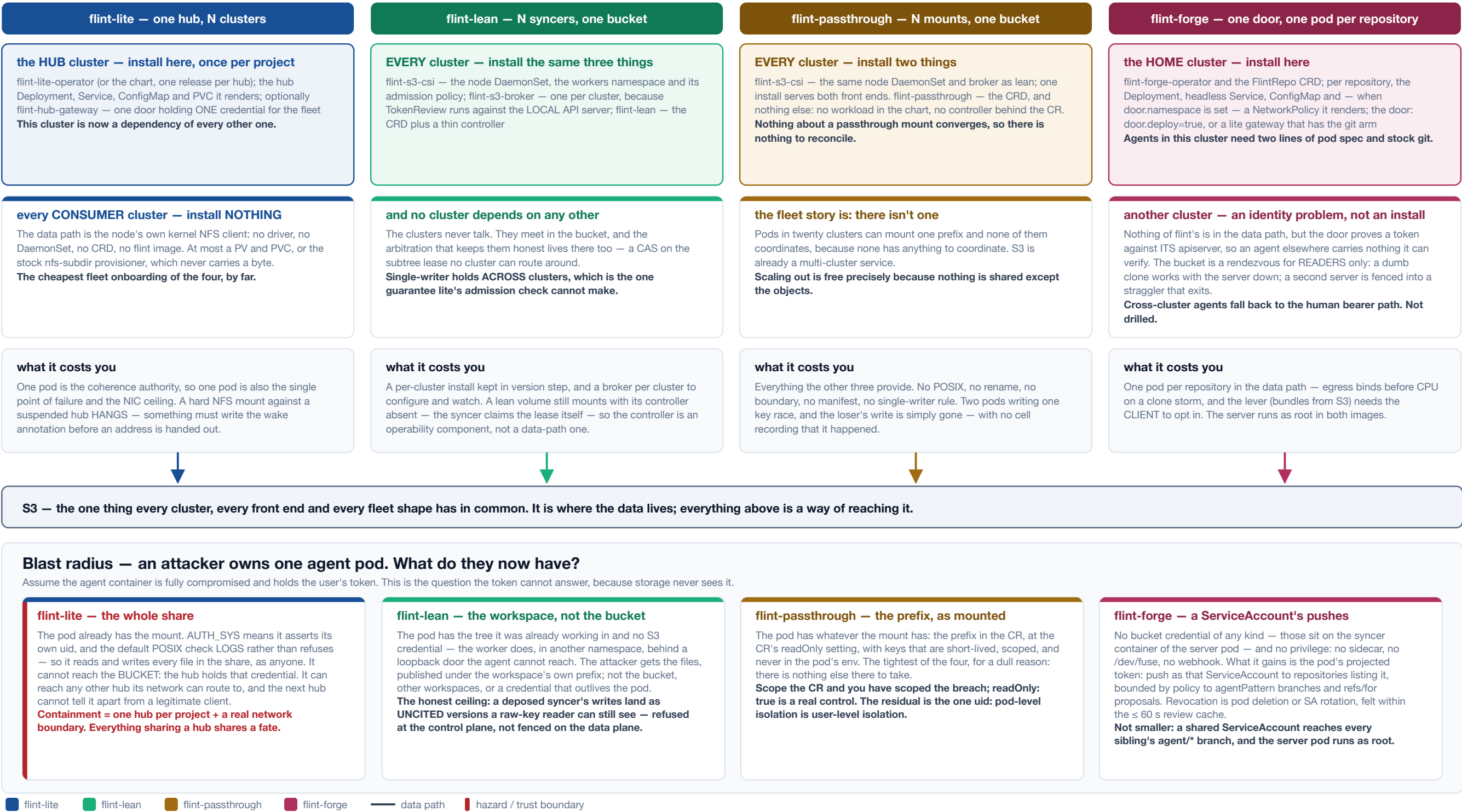
The levers are about taking the server out of the transfer. A thousand clones cost about 130 CPU-seconds but 43 GB from one NIC, so egress binds long before CPU does; measured on EC2 at 1,000 clones, the server's egress was 5.7 MiB with clone bundles advertised and the client opted in (`transfer.bundleURI=true`), against 40,409 MiB with the opt-in off — the storm falsifier's own arms; phase 0's 100-clone comparison put Gitea, the buy option, at 4,019 MiB, level with forge's controls. LFS goes the same way — the batch API runs in the syncer, which has the bucket credentials the door deliberately does not, and answers with presigned URLs, so a checkpoint never crosses the pod. Idle is one rung: no pushes for `suspendAfterSecs` and the replica count goes to zero; the door arms a wake annotation on the CR and holds the request, which on the cluster meant a clone through a parked repository completed in 11 s on a pod the request itself created. The export publishes a ref's tree as a real lean workspace by running the shipped `flint-sync` over it, after the report and never as a second writer of the snapshot.

The edges, stated as the drills left them. All eleven falsifiers ran on EC2 on 2026-09-04 and went green, two of them after finding a defect: a fresh repository could not create `main` at all, because the protection rule and the missing merge target closed on each other; and the export froze permanently once lean's own conditional upload parked on an etag it had not written. Nothing in-tree terminates TLS, so the pod's audience-bound token rides two cleartext hops inside the cluster; the `X-Remote-User` trust boundary is a NetworkPolicy the operator renders only when told where the door runs; the built idle clock counts pushes only, so read-only use is parked one threshold after its wake; and the server runs as root in both images. An export is a mirror that is never repaired — page 11.

The answer, for all four: it does not reach the data path. Every front end authenticates the WORKLOAD, never the human behind it.



Two practical questions for anyone running agents on more than one cluster: what do I have to install in each of them, and if one agent pod is fully compromised, what does the attacker now have? The four front ends answer both differently, and the cheapest to install is not the one with the smallest blast radius.



flint-lite has the cheapest fleet onboarding and the widest blast radius. Consumer clusters install nothing at all — the data path is the node's own kernel NFS client — but the hub cluster becomes a dependency of every other one, and a compromised pod already holds the mount. With AUTH_SYS and a log-only permission check, it reads and writes every file in the share as anyone. It cannot reach the bucket, because the hub holds that credential and the pod has none; that much is genuinely contained. Everything sharing a hub shares a fate, so size hubs by the blast radius you are willing to accept, not only by capacity.

flint-lean and flint-passthrough cost a per-cluster install and buy a much smaller radius. Each cluster runs the same CSI node driver and its own broker; no cluster depends on any other, and they meet only in the bucket — where the lean lease is a CAS no cluster can route around, which is the one guarantee lite's admission check cannot make. A compromised lean pod gets the tree it was already working in, published under its own prefix; it does not get the bucket, other workspaces, or a credential that outlives the pod. **The honest ceiling is that lean refuses a deposed writer at the control plane, not on the data plane:** its writes land as uncited versions that no coherent reader resolves, but a raw-key reader can still see them. Passthrough is the tightest, for a dull reason: there is nothing else there to take. The pod gets the prefix in the CR at the CR's readOnly setting, with short-lived scoped keys it never sees.

flint-forge installs in one cluster and makes the other clusters an identity problem rather than an install. The home cluster runs the operator, the door and one small pod per repository; agents there need stock git and two lines of pod spec. Nothing of flint's is in the data path elsewhere — but the door proves a token against its apiserver, so an agent in another cluster carries nothing it can verify, and falls back to the human bearer path. The bucket is a rendezvous for readers only: a dumb-protocol clone works with the server down, and a second server is fenced into a straggler that exits. A compromised forge agent gains no bucket credential and no privilege; it gains its ServiceAccount's push rights, within policy, revoked by pod deletion within the review cache. Not smaller: a shared ServiceAccount reaches every sibling's agent/* branch, and the server pod runs as root.

Underneath all four front ends there is one durable store, and it is S3. Everything above it — the hub's disk, the lean workspace, the FUSE mount, the forge clone and its server's local repository — is a way of reaching bytes whose home is the bucket. Two views of those bytes exist, with different contracts, and confusing them is the most common way to get hurt here.

The LIVE view — strong, and never in the bucket

a hub's port 2049 and its file API · a lean workspace's local disk · a forge clone's working tree, and the server's local repository
close-to-open · byte-range locks · atomic rename · no RPO lag
Reads and writes land on the working set, through whatever holds coherence for it — the hub process, the agent's own kernel, or git.
Who reads it: front doors, editors, agents — anything touching a tree that is being changed right now. The hub's live view offers one thing no real S3 can: assembly under a temp name completed by an ATOMIC RENAME under a deny-both OPEN. A lean or forge live tree is visible to exactly one pod.

The PUBLISHED view — the bucket, and the durable copy

RPO-consistent snapshots per subtree, made of untorn whole-file generations — or, for forge, a bare repository whose refs move as ONE CAS'd object.
whole objects · versioned or content-named · read-only against a live subtree
A reader never sees half a file or half a batch. It may well see a file from before your last write — that is the contract, not a defect. Anything that is not a mount reads this view as plain S3, with no flint server in the read path at all.
Do not write into a live subtree from the console: a hand-made PUT under a prefix a hub, syncer or server owns is overwritten at the next flush, reported as a conflict, or silently uncited.

Inside the prefix — the bytes, and the cells that make concurrency safe

Every cell below exists to answer one question that cannot be answered from the object listing alone.

flint-lite writes

<prefix>/<path>
the published generations
.flint/epoch
who is the hub right now? — a lease
.flint/manifest
what is the whole tree? — DR from one GET
.flint/owner

flint-lean writes

<prefix>/files/<path>
the workspace, file for file
.flint/lean/epoch · claim
who may write this subtree? — one holder, by CAS, across every cluster
.flint/lean/current
what does the last boundary contain?
.flint/lean/inbox · conflicts/

flint-passthrough writes

<prefix>/<key>
the objects, and nothing else
There is no control namespace here, and that is the entire feature.
A prefix owned by somebody else's tooling stays exactly as it was; flint claims no lease and leaves no trace. You also get no answer to any question on the left.

flint-forge writes

git/objects/pack/*
immutable, content-named — never overwritten
git/snapshot
the refs, packs, bundles and exported commit — ONE CAS'd pointer
git/epoch · git/claim
who is the server? whose prefix?
lfs/objects/<oid>
and, under its OWN prefix, the export

Reserved, and enforced

.flint is control state: a client file under it can never shadow a control object, and a NESTED .flint/ is another share's namespace — never tiered into this one. Both are tested, because both were once a way to overwrite a live share's lease.
The file-shaped formats are mutually convertible, and forge's export is one of them: choosing today does not strand data.

What “durable” actually means here — the recovery point, per front end, in the language of a hard pod kill

flint-lite — RPO is the flush cadence

The PVC is a working set the hub flushes on a timer; losing the disk is a REBUILD from the bucket, not a loss. A cold read hydrates on demand and both doors say so: NFS4ERR_DELAY, or 503.
Hibernation deletes the PVC, so it is VERIFIED, never assumed:
the operator scales back to one, polls until the hub reports rpoClean, and only then deletes. A share with no bucket reports rpoClean: null — a refusal, not a pass.

flint-lean — RPO is the last barrier

A hard kill loses at most floorSecs (default 60 s); a graceful stop loses nothing, because preStop runs a final barrier. A restart over a live tree RESUMES from its baseline.
Publish at a breakpoint and the RPO becomes zero, knowably:
echo > .flint/publish → publish.ack = ok
means the bytes are in S3 — the difference between “probably durable” and “durable, and I was told so”.

flint-passthrough — there is no RPO

Nothing is buffered on flint's behalf, so nothing can be lost by flint. A completed PUT is durable the instant S3 says so; an interrupted one did not happen.
The strongest durability story of the four — and strongest because flint is not in it.
The trade: a write is a whole new object, there is no rename to make it atomic against a reader, and two writers racing one key resolve as S3 resolves them — last writer wins, unrecorded.

flint-forge — RPO is the last acknowledged push

No path acknowledges a push the bucket does not hold: the pack is uploaded, the snapshot CAS lands, the ref transaction applies, and only THEN is ok reported. A crash between the CAS and the report fails the push at the client; the restart restores from the snapshot and passes fsck.
The unit is a push. Uncommitted work has no RPO —
git's contract, which forge keeps; a harness that wants one runs a snapshot script in its own pod, force-pushing refs/wip/<pod>.

flint-lite flint-lean flint-passthrough flint-forge hazard / trust boundary

The live view is strong and is never in the bucket. It is a hub's port 2049 and file API, a lean workspace's local disk, or a forge clone's working tree: close-to-open, byte-range locks, atomic rename, no RPO lag. The published view is the bucket: RPO-consistent snapshots per subtree, made of untorn whole-file generations — or, for forge, a bare repository whose refs move as one CAS'd object, so a reader never sees half a batch either. A reader of the bucket never sees half a file, and may well see a file from before your last write — that is the contract, not a defect. **Do not write into a live subtree from the console:** a hand-made PUT under a prefix a hub, syncer or server owns is overwritten at the next flush, reported as a conflict, or silently uncited — and under an export prefix, as page 11 shows, it simply stands.

Every control cell exists to answer a question the object listing cannot. Who is the hub right now, and may I become it — the epoch lease. What does the last coherent boundary contain — the manifest, which also makes disaster recovery a single GET rather than a crawl. Who owns this prefix, and did state loss change that — the claim and owner cells. Forge's `git/snapshot` answers all three of its own at once — the refs, the packs, the bundles, the exported commit — and its `git/epoch` is the same kind of lease under a different key, which is exactly why two products on one prefix never contend. Passthrough writes none of them, which is why it can be pointed at a prefix another system already owns and leaves no trace when the pod goes away; it also gets no answer to any of those questions.

The recovery point differs, and it is worth stating in the language of a hard pod kill. Lite loses up to the flush cadence, and its PVC is a cache — losing the disk is a rebuild. Lean loses at most `floorSecs` (default 60 s), zero on a graceful stop, and zero-and-acknowledged if the agent declares a boundary. Passthrough has no recovery point at all, because nothing is buffered on flint's behalf: a completed PUT is durable and an interrupted one did not happen. Forge's is the last acknowledged push — no path acknowledges a push the bucket does not hold, which eight mid-push pod kills on EC2 left standing with `fsck --strict` clean — and uncommitted work has none, which is git's contract.

Seven questions, asked of each front end in turn: how a writer proves who it is, what protects the wire, what a merely-reachable peer can do, what keeps tenants apart, whether two sessions on one project can hold different rights, what an attacker who owns an agent pod gains, and whether the posture fails closed under operator error. Every cell is a claim the pages before this one already made, or a verified cell of the approach radar; none is a score.

dimension	flint-lite	flint-lean	flint-passthrough	flint-forge
Who a writer must prove it is before storage accepts an operation	Nobody, on port 2049. AUTH_SYS: the client asserts its own uid and the server takes it. Identity is self-declared, not proven; the file API's bearer is per-share. krb5 exists in the server and is not surfaced by the chart.	The pod's ServiceAccount, online. kubelet-minted, pod-bound, audience s3.csi.chert.us; TokenReview at the broker, a registration the pod cannot self-mint, and the CR's consumers list. The agent container holds no key at all.	The pod's ServiceAccount, online. The identical chain: same broker, same TokenReview, same registration binding, same consumers gate, same audit line. Inside the pod every process is one uid.	The pod's ServiceAccount, at the door. A projected token, audience forge.chert.us, as the Basic password; TokenReview per session, cached ≤ 60 s by hash. The principal is the SA, which many pods share — agentPattern bounds a branch name, not its owner.
What protects the wire confidentiality and integrity in transit	Cleartext RPC. sec=sys carries no per-user authentication and no encryption; RPC-with-TLS is prospective. Reachability is the boundary, so draw it at the network layer — peering, security groups, a gateway.	Nothing of flint's crosses the network. The agent writes local disk; the worker publishes to the S3 endpoint over its TLS. The credential door is a loopback socket on the node; the broker is reached in-cluster and answers keys, not bytes.	S3 over TLS, unmediated. mount-s3 talks to the endpoint directly; the FUSE descriptor crosses a node-local socket; keys arrive on a loopback door, never as pod env. No flint server is on the read path.	Two cleartext hops, as built. HTTPS at the door is a line in the diagram; nothing in-tree terminates it, and the git container's runner listens on 8080 with no TLS. The pod's audience-bound token rides both hops as a Basic password — a network observer inside the cluster holds a live credential. Server-to-S3 is TLS.
What a reachable peer can do with no credential at all	Open a full NFSv4.1 session. Port 2049 needs no credential: the defence is caps — state-table quota, slot cap, idle deadline, READ clamp — not authentication. A merely-reachable peer holds a stateful session.	Very little. No flint door sits on the data path; the broker refuses without a valid TokenReview, and S3 refuses unsigned requests. What it can flood is the broker's TokenReview against the local apiserver.	Very little. The same: the reachable service is the broker, and the store refuses unsigned requests. The node plugin's socket is kubelet's, not the network's.	A 401 before any wake — authentication precedes the repository's wake, so a peer cannot scale a parked repository up, and a refused token is cached. The lever it keeps: every novel bad token is one TokenReview; behind the door the runner sets no body limit and no timeout of its own (the door's idle bound is the one cut), and a wake holds a door slot up to 180 s.
What keeps tenants apart reading or writing another's data	The network, and one hub per project. No auth on 2049 means the boundary is reachability, which networkPolicy cannot draw for remote clusters; one CR, one prefix, one coherence domain, and everything sharing a hub shares a fate.	The prefix, its claim and its lease. Keys are scoped to the workspace prefix; the claim refuses a foreign project on a reused prefix; the lease admits one writer and lives in the bucket, so it holds across clusters; consumers absent means deny.	The prefix and the CR. consumers decides which ServiceAccounts may mount it, keys are scoped to the bucket and prefix in the CR, and the CR lives in the pod's own namespace. Nothing finer than a pod: one uid per mount.	The repository. Its own pod, its prefix with a claim that refuses a foreign projectId, arbitration of overlapping prefixes at the operator — export prefixes included — consumers at the door, and a per-principal branch policy read by two enforcers. X-Remote-User is the boundary, and it is a NetworkPolicy: opt-in, and only where the CNI enforces it.
Can two sessions differ? in rights: one project, one reader, one writer	No — only a client-side ro mount. A read-only claim gets ro, and the CSI publish forces it over operator options; but the server authenticates nobody, so any pod that reaches 2049 mounts rw and asserts uid 0. enforcePermissions checks a claimed uid: defence against accident, not against a session that wants to write.	No. A workspace is one writer: the publish path hardcodes read-write, the CRD has no readOnly, and the credential is one key per project. A second lean pod on the same prefix does not become a reader — it waits to become the writer. The read-only session is a passthrough mount of files/, readOnly (page 11).	Yes, per pod. Two pods on one CR, one declared readOnly: --read-only on mount-s3 plus a read-only bind mount, and the pod never holds the key. The broker gates by consumers and hands both pods the SAME scope — one static key or one role ARN. The REST backend is told the ServiceAccount, so an external service can scope the key per principal; nothing in-tree does.	Yes, per ServiceAccount. consumers grants read; the branch policy decides writes per principal, in pre-receive and again in the syncer. A reader is listed in consumers with every ref protected and only the writer's SA in pushers; refs/for stays closed without mergeInto. Until a branches block exists, every consumer may push.
What an owned agent pod gains and how fast the gain is revoked	The whole share, as anyone. It has the mount, asserts any uid, and the permission check logs by default. Not the bucket — the hub holds that credential. Nothing per-client to revoke; containment is the network boundary.	Its own workspace, and nothing that outlives it. No S3 credential in the agent container in either deployment mode; the sidecar holds it, unprivileged. Revocation is killing the token or the pod. Ceiling: a deposited writer's uncited versions remain readable by a raw key.	The prefix, as mounted. Short-lived scoped keys it never sees, at the CR's readOnly setting; no sidecar, no privilege. Scope the CR and you have scoped the breach; the residual is the single uid.	Its ServiceAccount's push rights. No bucket credential — they land on the syncer container only — and no privilege. It can push as that SA to repositories listing it, within policy; revoked by pod deletion or SA rotation within the review cache. Ceiling: a shared SA reaches every sibling's agent/* branch, and the server runs as root.
Does it fail closed? secure defaults, refusal over silent degradation	No, by default. security.enforcePermissions is false — evaluate, log, allow — and even enforced the hub holds CAP_DAC_OVERRIDE. SECINFO now advertises AUTH_SYS first and AUTH_NONE last, pinned by a test, so a stock mount negotiates sec=sys.	Detection becomes refusal. A syncer that lost its lease answers refused-fenced rather than publishing; the plugin refuses to clean up a published tree; the chart refuses to render on credential misconfig; gated mode is refused without a lag bound.	Deny is the default. consumers absent means deny; the ValidatingAdmissionPolicy — not the namespace label — fences the workers namespace to the plugin's own node; a dead mounter strands the pod on ENOTCONN rather than serving stale bytes.	Closed where it counts, open on two defaults. An unparseable policy refuses; a MISSING pre-receive does not open the repository; a snapshot naming a missing pack refuses to serve. No door.namespace renders no NetworkPolicy, and an absent branches block is permissive: both documented, neither refuses.

Read it down a column. The identity chain lean and passthrough share is the strongest thing on the page, and forge borrows its shape at the door; the hub's is the weakest, and its boundary is drawn with the network. Every cell is the deck's own claim, made on the pages before this one, or the approach radar's verified Security cell.

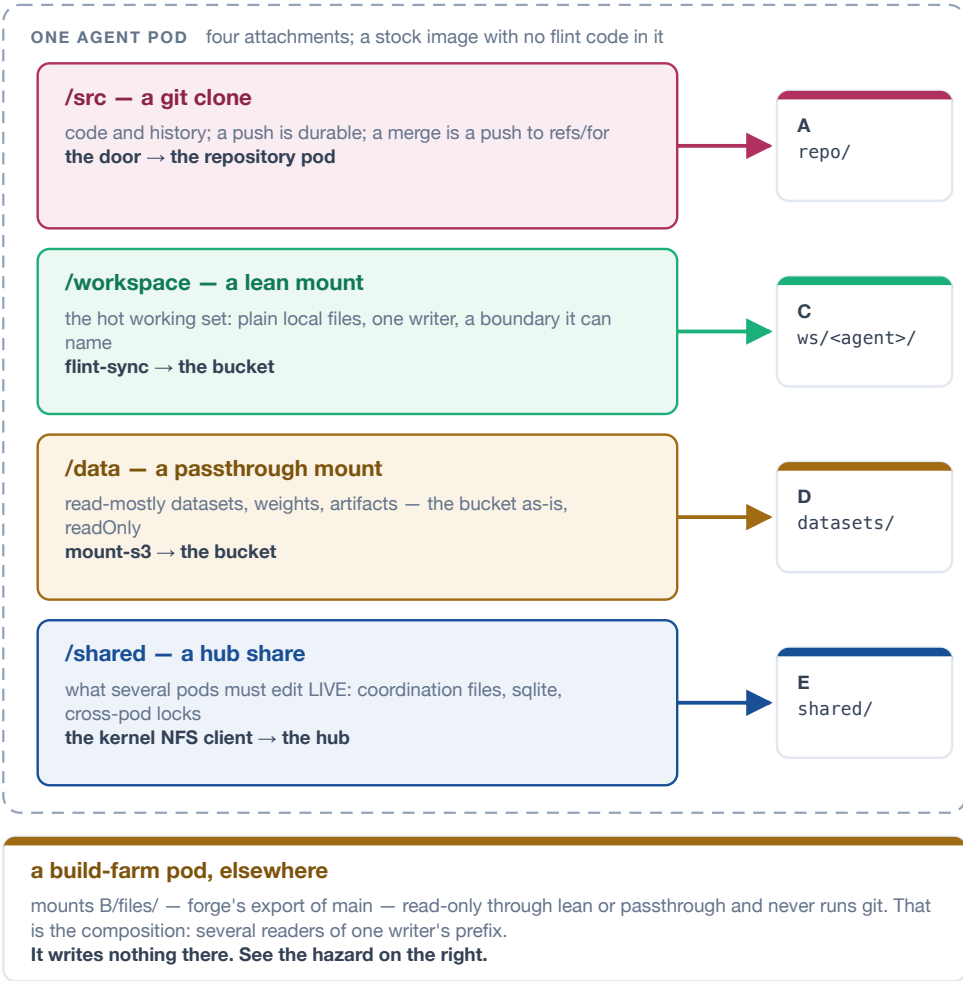
flint-lite flint-lean flint-passthrough flint-forge hazard / trust boundary

Read it down a column. The chain lean and passthrough share is the strongest thing on the page: the pod's ServiceAccount, kubelet-minted and pod-bound, verified *online* by a broker that also demands a registration the pod cannot self-mint and a **consumers** entry in the token's own namespace, exchanged for short-lived keys scoped to one prefix that the agent container never holds. Forge borrows the first half of that shape at its door — the same TokenReview, cached at most 60 s — and then differs in what happens next: the principal is carried to the server as a header, which makes the network between door and server a trust boundary, and nothing in-tree terminates TLS, so the token itself rides both hops in the clear. The hub's is the weakest: port 2049 proves nothing, the permission check logs by default, and the boundary has to be drawn with the network, which **networkPolicy** cannot do for remote clusters.

None of the four knows the human. A user's read-only role becomes a read-only session only when whatever launches the pod picks the ServiceAccount, or sets the mount flag, from that role. Kubernetes lets anyone who can create pods in a namespace use any ServiceAccount in it, so the reader and the writer need separate namespaces or an admission policy that binds user to ServiceAccount; flint never sees that decision. Given it, the row above holds: two sessions with different rights on one project work on forge and on passthrough, are a mount flag the server cannot back on lite, and on lean are a composition with a passthrough reader. Three of the four were put on the wire on 2026-09-05: on forge a reader and a writer on one repository, refused and allowed per ServiceAccount, with the **X-Remote-User** boundary shown to hold only behind the operator's NetworkPolicy — delete it and a forged header merges; on passthrough two pods on one mount, separated only by the read-only flag, both drawing the same key; and on lite a krb5p mount that proves the user and enforces none of their rights, even in enforce mode. Lean's single writer is by construction, undrilled.

Two defaults are worth knowing before the first install. On lite, **security.enforcePermissions** is **false**, and a stock mount negotiates **sec=sys**; that is identity, not enforcement. On forge, a chart with no **door.namespace** renders no NetworkPolicy, so the **X-Remote-User** boundary is silently absent, and a repository with no **branches block** is permissive, so any listed consumer may move **main**. Both are documented; neither refuses. Lean and passthrough fail closed where it counts — a syncer that lost its lease answers **refused-fenced** rather than publishing, an absent **consumers** list means deny, and the workers namespace is fenced by an admission policy rather than a label — and forge does too on its hooks: an unparseable policy refuses, a missing **pre-receive** does not open the repository, and a snapshot naming a pack the bucket lacks refuses to serve.

The front ends are not alternatives to pick between once. A workload has several kinds of data, each kind has a best door, and one pod can hold all of them at once — a forge clone, a lean workspace, a passthrough mount, a hub share — because each lands in its own prefix of one bucket. This page states the rule that keeps that safe, and what the composition drills of 2026-09-04 found when the rule was tested rather than assumed.



Ten dimensions, four columns. Read it down a column rather than across a row: each front end is coherent with itself, and the rows that look like weaknesses are usually the price of a strength two rows up. Then read it against page 11, because the last row names a data class, not a team.

dimension	flint-lite	flint-lean	flint-passthrough	flint-forge
What the pod sees the shape of the thing	a shared NFS mount served by one hub pod that every client reaches over the network — one tree, many pods, live	plain files on local disk, checked out before the first line runs and published back on a boundary — one pod's own tree	an S3 prefix presented as files by Mountpoint for S3 — objects, with a directory-shaped view over them	a git remote. The working tree is a local clone the pod owns; history, branches and a merge policy live at the server
POSIX fidelity what actually works	Full, and shared. byte-range locks, atomic rename, O_EXCL — across pods and clusters	Full, and local. a real filesystem, so git, sqlite and hard links work — for one pod	None to speak of. no rename, no append, no in-place write — at any setting, deliberately	Full, and local — for tracked files. the clone is a real disk; what is durable is what is committed
Concurrent writers per tree	Many, coordinated. the only front end where two pods can edit one directory and both be right	Exactly one, enforced. a second syncer is REFUSED, not merged — several agents integrate through git instead	Many, uncoordinated. two writers on one key race; the loser's write is gone and nothing records it	Many, serialised by the server. one syncer per repository; a stale push is refused by name; a merge is a push to refs/for/<target>
What a reader sees the consistency contract	the live tree, strongly — close-to-open, no RPO lag, through the coherence authority	the last PUBLISHED boundary, never your last write; gated mode makes it all-or-nothing	whatever S3 currently holds, with S3's own consistency and nothing added	on fetch, every acknowledged push — served by the write authority, never stale, never from S3; between fetches, a snapshot with no signal
Across clusters the fleet shape	ONE hub, mounted from N clusters. Works unchanged — but needs unique client names, a network boundary and a keepalive	N independent syncers, one bucket. Clusters never talk; the lease lives in the bucket, so single-writer holds ACROSS clusters	N independent mounts, one bucket. There is no fleet problem, because nothing is shared to get wrong	one server per repository; the door proves tokens to ITS apiserver, so remote agents need the human bearer path. The bucket is a rendezvous for readers only — undrilled
Identity that is enforced the user's token reaches NONE of these	Nobody, on the data path. AUTH_SYS: the client asserts its own uid, and the POSIX check defaults to log-only	The POD's ServiceAccount. kubelet-minted, pod-bound, verified ONLINE by TokenReview, checked against consumers	The POD's ServiceAccount. an identical chain to lean — same broker, same registration binding, same audit line	The POD's ServiceAccount, at the door. TokenReview per session, consumers per repository, then X-Remote-User to two enforcers of one policy
Isolating user A from B when both launch agents on several clusters	Give each project its own hub and prefix, and bound reachability outside Kubernetes. Everything sharing a hub shares a fate.	Give each user their own workspace prefix. Keys are scoped to it, the lease fences it, and consumers says who may mount it	Give each user their own CR and pod. The mount presents ONE uid, so pod-level isolation IS user-level isolation	Give each project its own repository and each agent its own branch pattern; a shared ServiceAccount reaches every sibling's agent/* branch
Installed per cluster in a consumer cluster	Nothing. the node's own kernel NFS client is the data path; at most a PV and PVC	the s3.csi.chert.us driver, a broker and the lean CRD + thin controller — in EVERY cluster; the broker must be local	the same driver and broker, and the CRD — in every cluster. One CSI install serves lean and passthrough together	Nothing in the data path. stock git plus two lines of pod spec; the door and the servers live in the home cluster
Recovery point what a hard kill costs	the flush cadence. The PVC is a cache — losing it is a rebuild from S3, not a loss	the last barrier: ≤ floorSecs (default 60 s), or zero and acknowledged if you declare one	None — nothing is buffered. a completed PUT is durable; an interrupted one did not happen	the last acknowledged push — ok means the pack and the snapshot are in S3. Uncommitted work: none, by git's contract
Choose it for a data class, not a team	What several pods must edit LIVE. shared workspaces, cross-pod locking, sqlite, a dataset too big for a pod's disk	One agent's hot working set. a repo-sized tree, full POSIX at local speed, durable at breakpoints	The bucket, not a filesystem. read-mostly datasets, weights, artifacts, or a prefix another system already owns	Code, with history. durable on push, a merge policy the server enforces, many agents proposing to one main; large binaries in LFS

Read the table down a column, not across a row: each front end is coherent with itself, and a row that looks like a weakness is usually the price of a strength two rows up.

Then read the composition page: one workload usually has several of these data classes, and each has a best door. Because the file-shaped formats are common and forge exports one, a volume can change its mind later.

The short form. Choose **flint-lite** for what several pods must edit live — shared workspaces, cross-pod locking, sqlite, a dataset larger than a pod's disk, or anything needing a filesystem two pods can both see. Choose **flint-lean** for one agent's hot working set: a repo-sized tree that fits on disk, full POSIX at local speed, and a durable point the agent can name. Choose **flint-passthrough** when you want the bucket rather than a filesystem — read-mostly datasets, weights, artifacts, or a prefix another system already owns that you want visible inside a pod without changing it. Choose **flint-forge** for code with history: durable on push, a merge policy the server enforces, many agents proposing to one main, and large binaries in LFS.

The multi-cluster and security rows are the ones most often decided late and regretted. If one user's agents must be isolated from another's, none of the four will do it for you inside the data path: the unit of isolation is the prefix, and the thing that enforces it is a network boundary around a hub, a broker scoping short-lived keys to a CR, or a door and a branch policy around a repository. Decide the prefix layout first and the front ends second — plural, because a workload usually has several data classes and each has a best door. **And because the file-shaped formats are common and forge exports one, a volume can change its mind later** — which is the strongest argument for starting with the cheapest bargain that works rather than the most capable one.